DEVOPS PRACTICAL LAB

Student Name: Mohammed Sanjid

Project Title: DevOps Practical Lab: Application Development, Containerization, CI/CD, Infrastructure as Code, and Observability

Environment: Linux virtual machine, Python, Docker, Jenkins, GitHub Actions, Terraform, Prometheus, Alertmanager, and Grafana

1. Introduction

This project demonstrates a practical DevOps workflow through six phases, beginning with application development and automated testing and progressing through containerization, CI/CD automation, infrastructure provisioning, and monitoring.

The project integrates several DevOps tools to demonstrate repeatable application builds, container management, infrastructure lifecycle operations, automated validation, resource monitoring, and alerting.

1.1 Objectives

- Develop and test a Python web application.
- Package the application using Docker and a multi-stage Dockerfile.
- Automate application validation and build tasks using Jenkins.
- Implement repository-based automation using GitHub Actions.
- Provision and manage infrastructure using Terraform.
- Collect host metrics using Node Exporter.
- Collect container metrics using cAdvisor.
- Configure Prometheus alert rules and Alertmanager.
- Build a Grafana dashboard for real-time infrastructure metrics.
- Verify the implementation using terminal outputs and browser screenshots.

1.2 Technologies Used

Technology	Purpose

Python and Flask	Web application
Pytest	Automated testing
Git and GitHub	Source control
Docker	Containerization
Docker Compose	Multi-container management
Jenkins	CI/CD automation
GitHub Actions	Repository workflow automation
Terraform	Infrastructure as Code
Prometheus	Metrics collection and alert evaluation
Node Exporter	Host metrics
cAdvisor	Container metrics
Alertmanager	Alert handling and routing
Grafana	Dashboard visualization
stress-ng	CPU stress testing
Linux and firewalld	System and network administration

2. Phase 1 — Application Development and Testing

2.1 Objective

The first phase established a Python web application and automated tests to verify its basic functionality.

2.2 Application File: `app.py`

The Flask application exposes a welcome page, a health endpoint, and a version endpoint.

```
from flask import Flask, jsonify

app = Flask(__name__)


@app.route("/")
def home():
    return "Welcome to the DevOps Practical Lab - Production and
Development!"
```

```
@app.route("/health")
def health():
    return jsonify(status="UP")


@app.route("/version")
def version():
    return jsonify(version="1.0.0")


if __name__ == "__main__":
    app.run(host="0.0.0.0", port=5000, debug=False)
```

Explanation

- `/` returns a welcome message.
- `/health` returns the application's health status.
- `/version` returns the application version.
- The application listens on port `5000`.
- Debug mode is disabled in the application startup configuration.

2.3 Dependency File: `requirements.txt`

```
Flask==3.1.0
```

This file specifies the Flask version required by the application.

2.4 Test File: `test_app.py`

```
from app import app


def test_home():
    client = app.test_client()
    response = client.get("/")
    assert response.status_code == 200
    assert b"Welcome to the DevOps Practical Lab" in response.data
```

```
def test_health():
    client = app.test_client()
    response = client.get("/health")
    assert response.status_code == 200
    assert response.get_json() == {"status": "UP"}


def test_version():
    client = app.test_client()
    response = client.get("/version")
    assert response.status_code == 200
    assert response.get_json() == {"version": "1.0.0"}
```

These tests verify the application's HTTP responses and the expected JSON values returned by the health and version endpoints.

2.5 Repository Exclusion File: `.gitignore`

The `.gitignore` file is used to exclude generated files and local development artifacts from version control.

Use the exact `.gitignore` content from the repository used in the practical. Its original contents are not available in the current record, so they are not reconstructed here.

2.6 Command Logs

Install application dependencies

```
pip install -r requirements.txt
```

This installs Flask and the application's declared dependencies.

Run the automated tests

```
pytest -v
```

This executes the test suite and displays individual test results.

Check the repository status

```
git status
```

This displays tracked changes and untracked files.

2.7 Result

The application and its automated tests established the foundation for subsequent containerization and CI/CD automation.

```
(venv) [root@workstation devops-practical-lab]# git remote -v && git branch -vv
origin  git@github.com:mohammedsanjid-stack/devops-practice-lab.git (fetch)
origin  git@github.com:mohammedsanjid-stack/devops-practice-lab.git (push)
  develop a67a45b [origin/develop] Update welcome message for development
* main    f80f58f [origin/main] Merge develop into main and resolve conflict
(venv) [root@workstation devops-practical-lab]#
```

```
(venv) [root@workstation devops-practical-lab]# git show --cc --format=fuller f80f58f
commit f80f58fd715a9e08c2444bf4f9f0863c7e83bdc7 (HEAD -> main, tag: v1.0.0, origin/main)
Merge: 5503049 a67a45b
Author:     Mohammed Sanjid <mohammedsanjid77@gmail.com>
AuthorDate: Thu Oct 22 02:24:46 2026 -0400
Commit:     Mohammed Sanjid <mohammedsanjid77@gmail.com>
CommitDate: Thu Oct 22 02:24:46 2026 -0400

    Merge develop into main and resolve conflict

diff --cc app.py
index 006420f,323ae7c..ca1ccee
--- a/app.py
+++ b/app.py
@@@ -1,3 -1,3 +1,4 @@@
++
  from flask import Flask, jsonify

  app = Flask(__name__)
@@@ -5,7 -5,7 +6,7 @@@

  @app.route("/")
  def home():
-     return "Welcome to the DevOps Practical Lab - Production!"
 -    return "Welcome to the DevOps Practical Lab - Development!"
++    return "Welcome to the DevOps Practical Lab - Production and Development!"


  @app.route("/health")
@@@ -13,5 -13,10 +14,11 @@@ def health()
(venv) [root@workstation devops-practical-lab]#
```

```
(venv) [root@workstation devops-practical-lab]# git log --graph --oneline --all --decorate
*   f80f58f (HEAD -> main, tag: v1.0.0, origin/main) Merge develop into main and resolve conflict
|\
| * a67a45b (origin/develop, develop) Update welcome message for development
| * b961364 Add version endpoint and unit test
* | 5503049 Update welcome message for production
|/
* 441eea8 Initial commit: add Flask app and unit tests
(venv) [root@workstation devops-practical-lab]#
```

3. Phase 2 — Docker Containerization

3.1 Objective

The second phase packaged the Python application into a Docker image to provide a consistent runtime environment.

The application was containerized and exposed through host port `5002`, mapped to container port `5000`.

3.2 Dockerfile

The project used a multi-stage Dockerfile.

```
FROM docker.io/library/python:3.11-slim AS builder

WORKDIR /build

COPY requirements.txt .

RUN pip install --no-cache-dir --prefix=/install -r requirements.txt


FROM docker.io/library/python:3.11-slim

WORKDIR /app

COPY --from=builder /install /usr/local
COPY app.py .

EXPOSE 5000

CMD ["python", "app.py"]
```

Explanation

- The builder stage installs Python dependencies.
- The runtime stage copies the installed packages from the builder.
- `app.py` is copied into the application directory.
- Port `5000` is declared as the container port.

- The application starts using `python app.py`.

The multi-stage build separates dependency installation from the final runtime image.

3.3 Command Logs

Build the application image

```
docker build -t devops-app:1.0 .
```

This builds the image from the Dockerfile and assigns the tag `devops-app:1.0`.

Verify the image

```
docker images
```

This lists the available Docker images.

Run the application container

The application container was named `devops-app`.

```
docker run -d --name devops-app -p 5002:5000 devops-app:1.0
```

This command creates a detached container and maps host port `5002` to container port `5000`. Run it only when the container does not already exist.

Verify running containers

```
docker ps
```

This displays running containers, their images, port mappings, and status.

The recorded application container was:

```
NAME: devops-app
IMAGE: devops-app:1.0
PORT MAPPING: 5002:5000
```

Test the application endpoint

```
curl -I http://localhost:5002
```

This checks the HTTP response headers returned by the application.

Inspect networks and volumes

```
docker network ls
docker volume ls
```

These commands list Docker networks and volumes available in the environment.

3.4 Docker Compose and Supporting Services

The lab environment also contained a Redis container named `redis-db`, using the `redis:7-alpine` image.

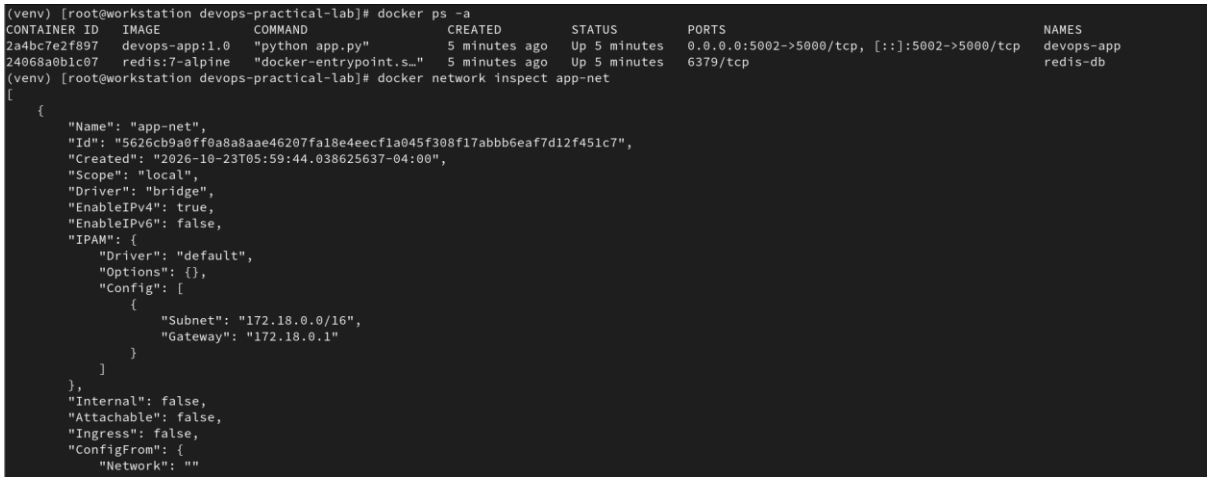
The confirmed running containers included:

- `devops-app`, exposed on host port `5002`.
- `redis-db`, running Redis on its default container port `6379`.

The exact Phase 2 Compose YAML and database configuration are not available in the current record. Insert the original file from the Phase 2 working directory if it was used. Do not replace it with a newly reconstructed configuration.

3.5 Result

The Flask application was containerized, and the running application was exposed through host port `5002`. Docker commands were used to inspect the image, container, networks, and volumes.



```
(venv) [root@workstation devops-practical-lab]# docker ps -a
CONTAINER ID   IMAGE          COMMAND                  CREATED         STATUS         PORTS                                                  NAMES
2a4bc7e2f897   devops-app:1.0   "python app.py"          5 minutes ago   Up 5 minutes   0.0.0.0:5002->5000/tcp, [::]:5002->5000/tcp   devops-app
24068a0b1c07   redis:7-alpine   "docker-entrypoint.s…"   5 minutes ago   Up 5 minutes   6379/tcp                                               redis-db
(venv) [root@workstation devops-practical-lab]# docker network inspect app-net
[
    {
        "Name": "app-net",
        "Id": "5626cb9a0ff0a8a8aae46207fa18e4eecf1a045f308f17abbb6eaf7d12f451c7",
        "Created": "2026-10-23T05:59:44.038625637-04:00",
        "Scope": "local",
        "Driver": "bridge",
        "EnableIPv4": true,
        "EnableIPv6": false,
        "IPAM": {
            "Driver": "default",
            "Options": {},
            "Config": [
                {
                    "Subnet": "172.18.0.0/16",
                    "Gateway": "172.18.0.1"
                }
            ]
        },
        "Internal": false,
        "Attachable": false,
        "Ingress": false,
        "ConfigFrom": {
            "Network": ""
```




```
(venv) [root@workstation devops-practical-lab]# docker run --rm --network app-net redis:7-alp
ine redis-cli -h redis-db -p 6379 ping
PONG
(venv) [root@workstation devops-practical-lab]#
```

```
(venv) [root@workstation devops-practical-lab]# docker images
                                                                    i Info →  U In Use
IMAGE            ID             DISK USAGE   CONTENT SIZE   EXTRA
devops-app:1.0   9aceeb162bda         191MB         47.3MB   U
redis:7-alpine   57e4255cc971        57.3MB         16.7MB   U
(venv) [root@workstation devops-practical-lab]# docker history devops-app:1.0
IMAGE          CREATED        CREATED BY                                      SIZE      COMMENT
9aceeb162bda   16 hours ago   CMD ["python" "app.py"]                         0B        buildkit.dockerfile.v0
<missing>      16 hours ago   EXPOSE [5000/tcp]                               0B        buildkit.dockerfile.v0
<missing>      16 hours ago   COPY app.py . # buildkit                        4.1kB     buildkit.dockerfile.v0
<missing>      16 hours ago   COPY /install /usr/local # buildkit             5.73MB    buildkit.dockerfile.v0
<missing>      16 hours ago   WORKDIR /app                                    0B        buildkit.dockerfile.v0
<missing>      3 days ago     CMD ["python3"]                                 0B        buildkit.dockerfile.v0
<missing>      3 days ago     RUN /bin/sh -c set -eux;  for src in idle3 p…   0B        buildkit.dockerfile.v0
<missing>      3 days ago     RUN /bin/sh -c set -eux;   savedAptMark="$(a…   47.6MB    buildkit.dockerfile.v0
<missing>      3 days ago     ENV PYTHON_SHA256=bfb74ad39efae27cda510f134a…   0B        buildkit.dockerfile.v0
<missing>      3 days ago     ENV PYTHON_VERSION=3.11.17                      0B        buildkit.dockerfile.v0
<missing>      3 days ago     ENV GPG_KEY=A035C8C19219BA821ECEA86B64E628F8…   0B        buildkit.dockerfile.v0
<missing>      3 days ago     RUN /bin/sh -c set -eux;  apt-get update;  a…   4.35MB    buildkit.dockerfile.v0
<missing>      3 days ago     ENV LANG=C.UTF-8                                0B        buildkit.dockerfile.v0
<missing>      3 days ago     ENV PATH=/usr/local/bin:/usr/local/sbin:/usr…   0B        buildkit.dockerfile.v0
<missing>      3 days ago     # debian.sh --arch 'amd64' out/ 'trixie' '@1…   86.2MB    debuerreotype 0.17
(venv) [root@workstation devops-practical-lab]#
```

4. Phase 3 — Jenkins CI/CD Pipeline

4.1 Objective

The third phase introduced Jenkins pipeline automation. A Jenkinsfile defined the stages used to retrieve source code, prepare the build environment, and validate the application.

4.2 Jenkinsfile

The pipeline was defined using declarative pipeline syntax.

```
pipeline {
    agent any

    stages {
        stage('Checkout') {
            steps {
                checkout scm
            }
        }

        stage('Build') {
            steps {
                echo 'Preparing the application build'
                sh 'python3 --version'
            }
        }

        stage('Test') {
            steps {
                sh 'python3 -m py_compile app.py test_app.py'
            }
        }
    }

    post {
        success {
            echo 'Pipeline completed successfully'
        }

        failure {
            echo 'Pipeline failed'
        }
    }
}
```

Explanation

- **Checkout:** Retrieves the source code from the configured source-control repository.
- **Build:** Checks the Python interpreter available to the Jenkins agent.
- **Test:** Compiles the application and test files to identify Python syntax errors.
- **Post actions:** Print a completion message based on the pipeline result.

The `py_compile` command validates syntax; it does not execute the full Pytest suite.

4.3 Command Logs

Review the Jenkinsfile

```
cat Jenkinsfile
```

This displays the pipeline definition.

Verify Python

```
python3 --version
```

This prints the installed Python version.

Validate Python syntax

```
python3 -m py_compile app.py test_app.py
```

This checks the syntax of the application and test files.

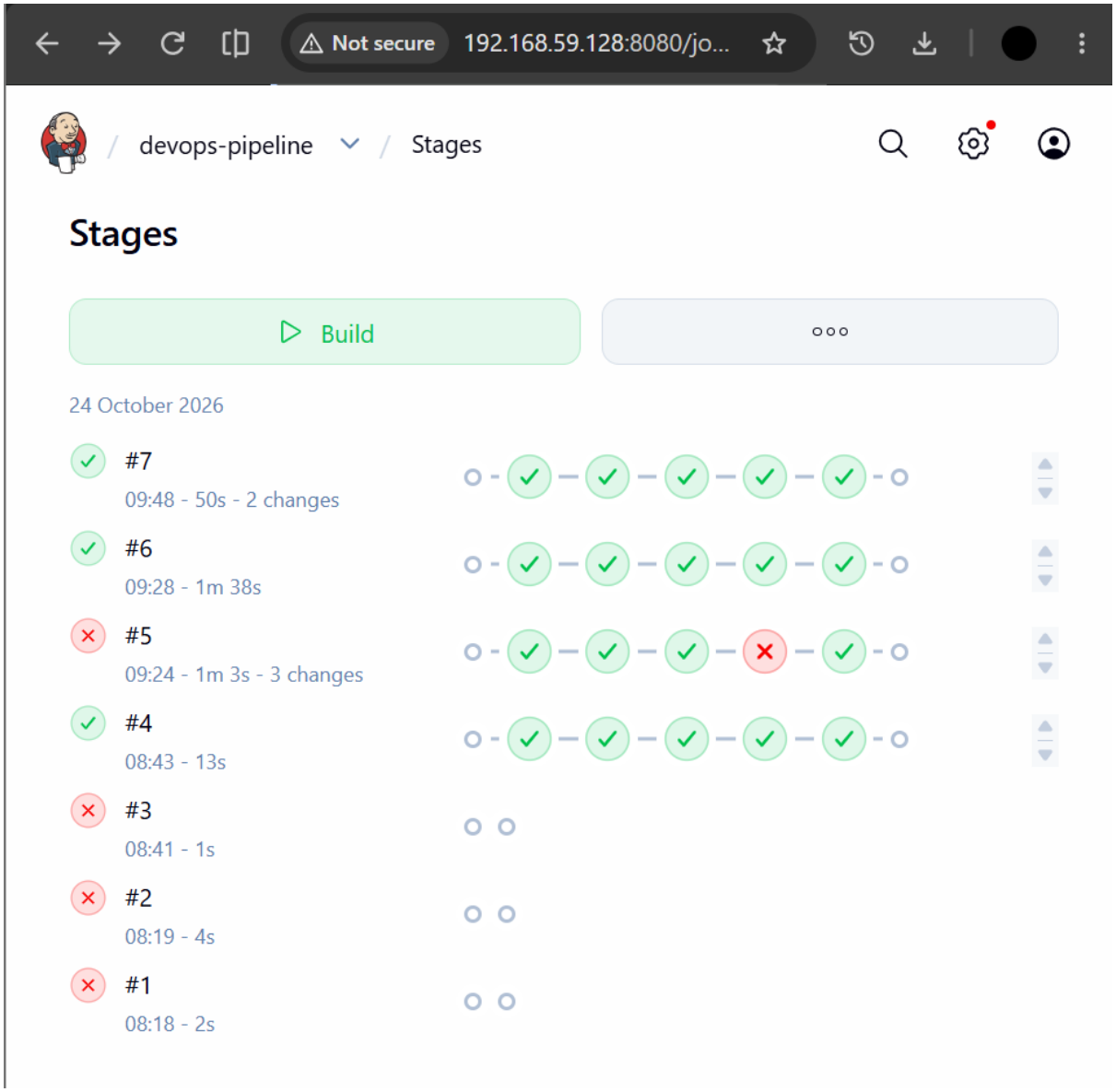
Verify Docker availability when required

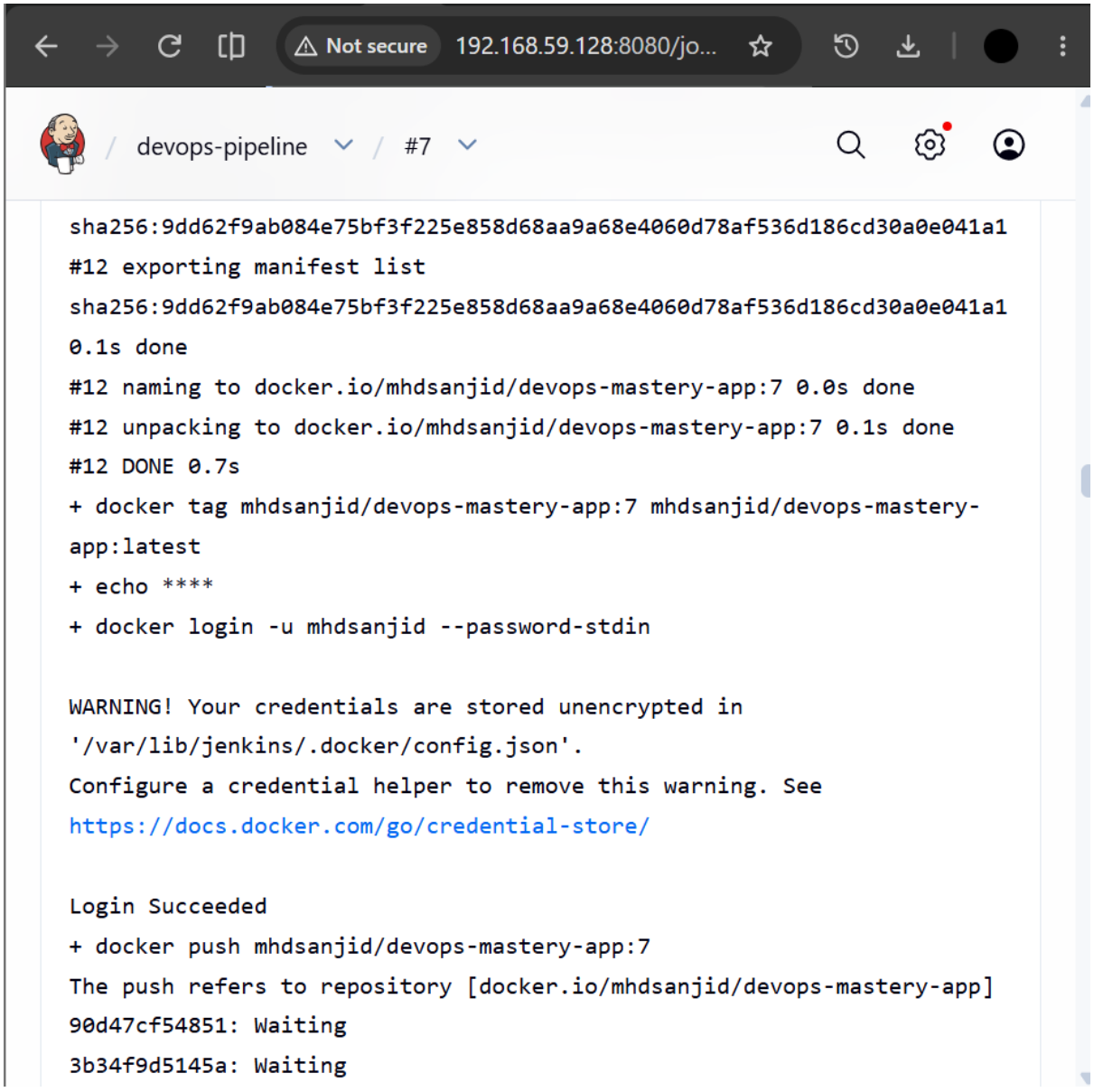
```
docker version
```

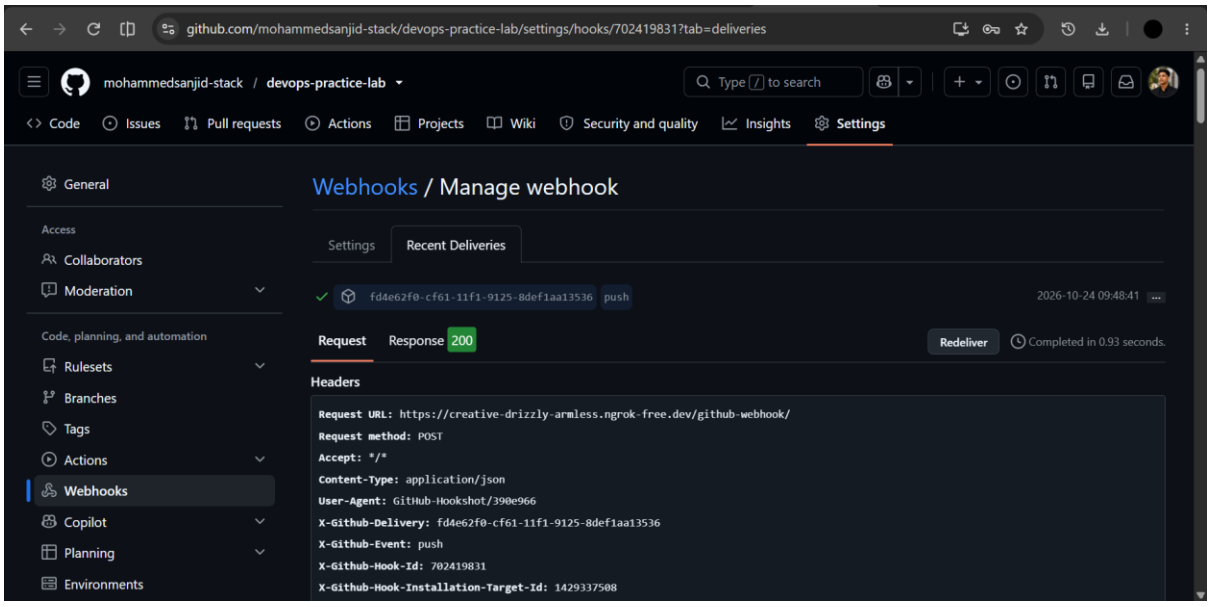
This displays Docker client and server information.

4.4 Result

Jenkins provided a repeatable, version-controlled definition of the application validation workflow.


Not secure
192.168.59.128:8080/jo...
devops-pipeline
Stages
Stages
Build
24 October 2026
#7
09:48 - 50s - 2 changes
#6
09:28 - 1m 38s
#5
09:24 - 1m 3s - 3 changes
#4
08:43 - 13s
#3
08:41 - 1s
#2
08:19 - 4s
#1
08:18 - 2s


Not secure
192.168.59.128:8080/jo...
devops-pipeline
#7
sha256:9dd62f9ab084e75bf3f225e858d68aa9a68e4060d78af536d186cd30a0e041a1
#12 exporting manifest list
sha256:9dd62f9ab084e75bf3f225e858d68aa9a68e4060d78af536d186cd30a0e041a1
0.1s done
#12 naming to docker.io/mhdsanjid/devops-mastery-app:7 0.0s done
#12 unpacking to docker.io/mhdsanjid/devops-mastery-app:7 0.1s done
#12 DONE 0.7s
+ docker tag mhdsanjid/devops-mastery-app:7 mhdsanjid/devops-mastery-app:latest
+ echo ****
+ docker login -u mhdsanjid --password-stdin
WARNING! Your credentials are stored unencrypted in '/var/lib/jenkins/.docker/config.json'.
Configure a credential helper to remove this warning. See
https://docs.docker.com/go/credential-store/
Login Succeeded
+ docker push mhdsanjid/devops-mastery-app:7
The push refers to repository [docker.io/mhdsanjid/devops-mastery-app]
90d47cf54851: Waiting
3b34f9d5145a: Waiting


github.com/mohammedsanjid-stack/devops-practice-lab/settings/hooks/702419831?tab=deliveries
mohammedsanjid-stack / devops-practice-lab
Type / to search
Code
Issues
Pull requests
Actions
Projects
Wiki
Security and quality
Insights
Settings
General
Access
Collaborators
Moderation
Code, planning, and automation
Rulesets
Branches
Tags
Actions
Webhooks
Copilot
Planning
Environments
Webhooks / Manage webhook
Settings
Recent Deliveries
fd4e62f0-cf61-11f1-9125-8def1aa13536
push
2026-10-24 09:48:41
Request
Response 200
Redeliver
Completed in 0.93 seconds.
Headers
Request URL: https://creative-drizzly-armless.ngrok-free.dev/github-webhook/
Request method: POST
Accept: */*
Content-Type: application/json
User-Agent: GitHub-Hookshot/390e966
X-Github-Delivery: fd4e62f0-cf61-11f1-9125-8def1aa13536
X-Github-Event: push
X-Github-Hook-Id: 702419831
X-Github-Hook-Installation-Target-Id: 1429337508

5. Phase 4 — GitHub Actions CI/CD Workflow

5.1 Objective

The fourth phase implemented repository-based automation using GitHub Actions. The workflow was designed to test the application across multiple Python versions and build and publish the Docker image after successful tests.

5.2 Workflow File: `.github/workflows/ci-cd.yml`

The workflow used a Python version matrix covering Python `3.10` and `3.12`. The Docker publishing job depended on successful tests and ran on pushes to the `main` branch.

The following is a structured representation of the workflow requirements recorded for the lab. Verify it against the original repository file before submitting it as the exact executed YAML.

```
name: CI/CD Pipeline

on:
  push:
    branches:
      - main
  pull_request:

jobs:
  test:
    runs-on: ubuntu-latest

    strategy:
      matrix:
        python-version:
          - "3.10"
          - "3.12"

    steps:
      - name: Checkout repository
        uses: actions/checkout@v4
```

```
      - name: Set up Python
        uses: actions/setup-python@v5
        with:
          python-version: ${{ matrix.python-version }}

      - name: Install dependencies
        run: |
          python -m pip install --upgrade pip
          pip install -r requirements.txt
          pip install pytest

      - name: Run tests
        run: pytest -v

  build-and-push:
    needs: test
    if: github.event_name == 'push' && github.ref ==
'refs/heads/main'
    runs-on: ubuntu-latest

    steps:
      - name: Checkout repository
        uses: actions/checkout@v4

      - name: Set up Docker Buildx
        uses: docker/setup-buildx-action@v3

      - name: Log in to Docker Hub
        uses: docker/login-action@v3
        with:
          username: ${{ secrets.DOCKER_USERNAME }}
          password: ${{ secrets.DOCKER_PASSWORD }}

      - name: Build and push image
        uses: docker/build-push-action@v6
        with:
          context: .
          push: true
          tags: mhdsanjid/devops-mastery-app:${{ github.sha }}
```

Explanation

- The workflow runs on pushes to `main` and pull requests.
- The `test` job executes in a matrix covering Python 3.10 and Python 3.12.
- Dependencies are installed before the Pytest suite runs.
- The `build-and-push` job depends on successful completion of the `test` job.
- The image publishing job runs only on pushes to `main`.
- Docker Hub credentials are obtained through repository secrets.
- The image tag uses the GitHub commit SHA for build identification.

Security: Never place Docker Hub passwords, tokens, or access keys directly in the YAML file.

5.3 Command Logs

Review the workflow

```
cat .github/workflows/ci-cd.yml
```

Check repository status

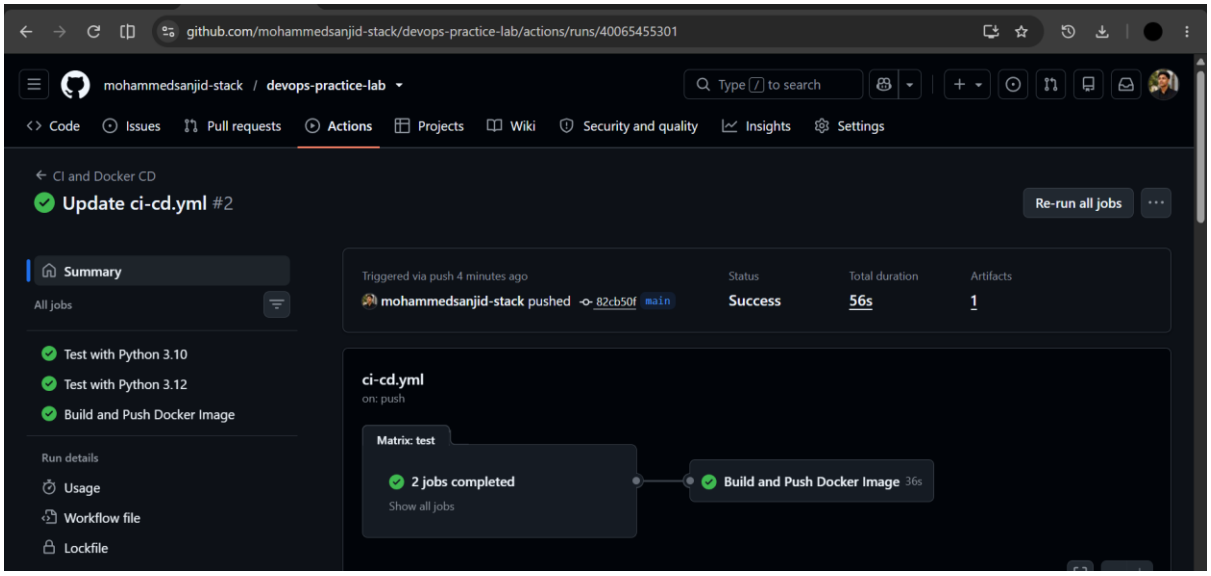
```
git status
```

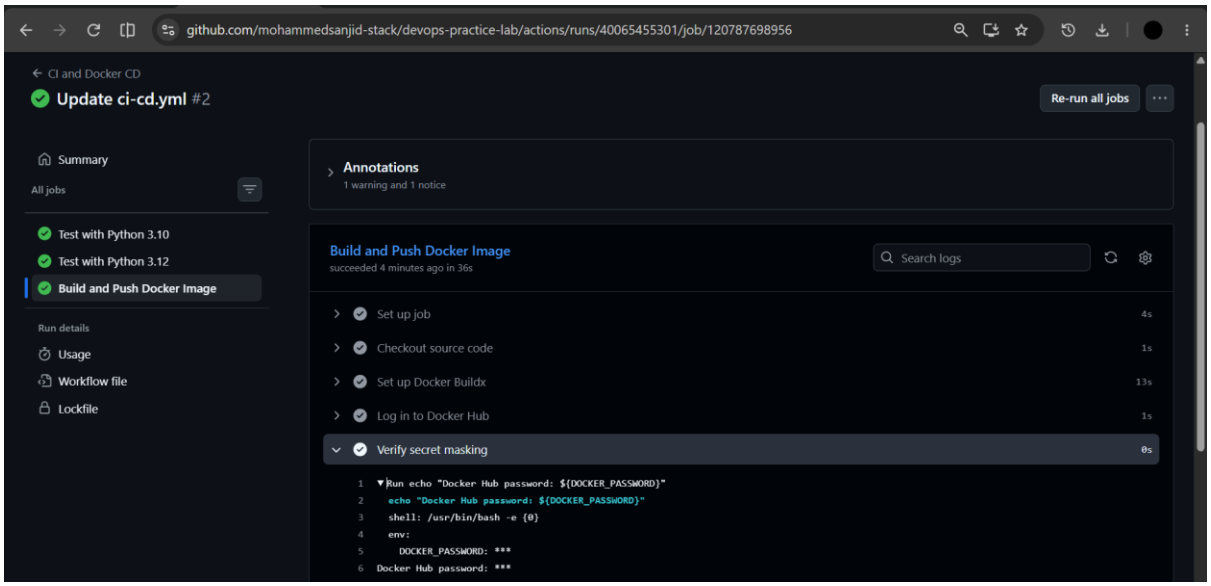
Review recent commits

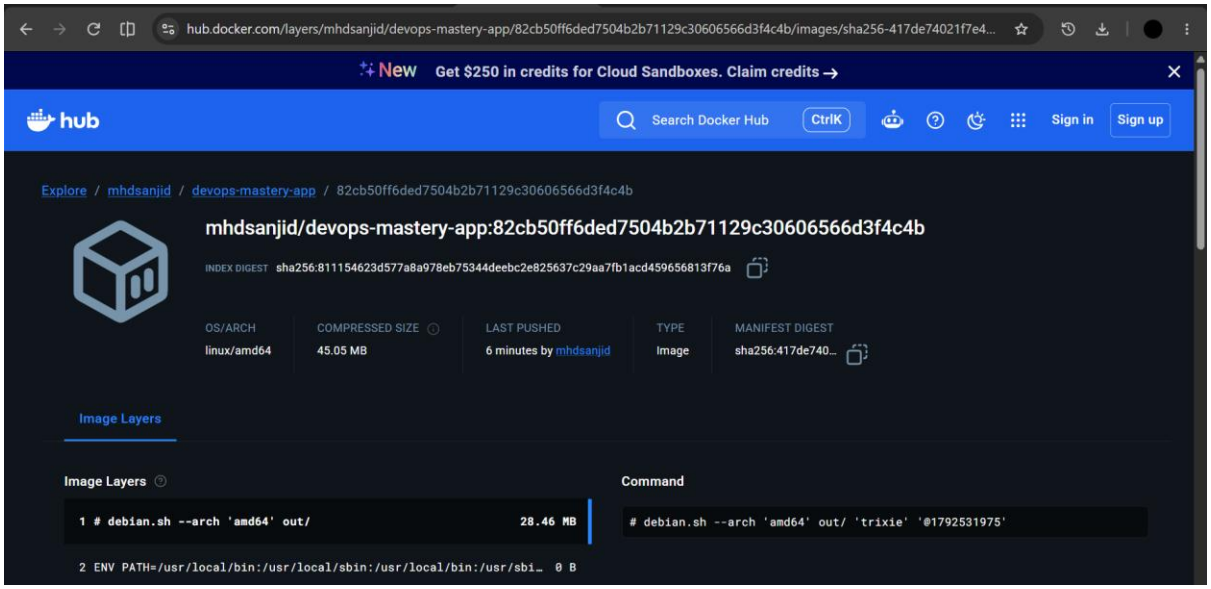
```
git log --oneline -5
```

5.4 Result

GitHub Actions provided automated testing across multiple Python versions and a conditional Docker image publishing workflow.


github.com/mohammedsanjid-stack/devops-practice-lab/actions/runs/40065455301
mohammedsanjid-stack / devops-practice-lab
Type / to search
Code
Issues
Pull requests
Actions
Projects
Wiki
Security and quality
Insights
Settings
CI and Docker CD
Update ci-cd.yml #2
Re-run all jobs
Summary
All jobs
Test with Python 3.10
Test with Python 3.12
Build and Push Docker Image
Run details
Usage
Workflow file
Lockfile
Triggered via push 4 minutes ago
mohammedsanjid-stack pushed 82cb50f main
Status
Success
Total duration
56s
Artifacts
1
ci-cd.yml
on: push
Matrix: test
2 jobs completed
Show all jobs
Build and Push Docker Image 36s


github.com/mohammedsanjid-stack/devops-practice-lab/actions/runs/40065455301/job/120787698956
CI and Docker CD
Update ci-cd.yml #2
Re-run all jobs
Summary
All jobs
Test with Python 3.10
Test with Python 3.12
Build and Push Docker Image
Run details
Usage
Workflow file
Lockfile
Annotations
1 warning and 1 notice
Build and Push Docker Image
succeeded 4 minutes ago in 36s
Search logs
Set up job 4s
Checkout source code 1s
Set up Docker Buildx 13s
Log in to Docker Hub 1s
Verify secret masking 0s
Run echo "Docker Hub password: ${DOCKER_PASSWORD}"
echo "Docker Hub password: ${DOCKER_PASSWORD}"
shell: /usr/bin/bash -e {0}
env:
DOCKER_PASSWORD: ***
Docker Hub password: ***


hub.docker.com/layers/mhdsanjid/devops-mastery-app/82cb50ff6ded7504b2b71129c30606566d3f4c4b/images/sha256-417de74021f7e4...
New
Get $250 in credits for Cloud Sandboxes. Claim credits
hub
Search Docker Hub
CtrlK
Sign in
Sign up
Explore / mhdsanjid / devops-mastery-app / 82cb50ff6ded7504b2b71129c30606566d3f4c4b
mhdsanjid/devops-mastery-app:82cb50ff6ded7504b2b71129c30606566d3f4c4b
INDEX DIGEST sha256:811154623d577a8a978eb75344deebc2e825637c29aa7fb1acd459656813f76a
OS/ARCH
linux/amd64
COMPRESSED SIZE
45.05 MB
LAST PUSHED
6 minutes by mhdsanjid
TYPE
Image
MANIFEST DIGEST
sha256:417de740...
Image Layers
Image Layers
1 # debian.sh --arch 'amd64' out/ 28.46 MB
2 ENV PATH=/usr/local/bin:/usr/local/sbin:/usr/local/bin:/usr/sbi… 0 B
Command
debian.sh --arch 'amd64' out/ 'trixie' '@1792531975'

6. Phase 5 — Infrastructure as Code with Terraform

6.1 Objective

The fifth phase demonstrated Infrastructure as Code using Terraform. The work included initializing the Terraform project, reviewing plans, changing a web port variable, applying the updated configuration, inspecting the state, and destroying the Terraform-managed resources.

The working directory was:

`/opt/devops-practical-lab/phase5`

6.2 Terraform Project Structure

The project used the following configuration structure:

```
phase5/
├── main.tf
├── variables.tf
├── outputs.tf
└── modules/
    └── web_container/
        ├── main.tf
        ├── variables.tf
        └── outputs.tf
```

The main configuration uses a reusable module to define the web-container resource. Variables provide configurable inputs, while outputs expose useful resource information.

The exact contents of the Phase 5 Terraform files are not present in the available record. Preserve the original `main.tf`, `variables.tf`, `outputs.tf`, and module files from the actual lab rather than substituting reconstructed infrastructure definitions.

6.3 Command Logs

Step 1: Navigate to the working directory

```
cd /opt/devops-practical-lab/phase5
```

Step 2: Initialize Terraform

```
terraform init
```

This initializes the working directory and downloads required providers and modules.

Step 3: Inspect the initial plan

```
terraform plan
```

The recorded initial plan was:

```
Plan: 4 to add, 0 to change, 0 to destroy.
```

This showed the proposed creation of four resources.

Step 4: Change the host port

The web-container configuration was changed from host port `8081` to host port `8082`.

Run the plan again:

```
terraform plan
```

The recorded result was:

```
Plan: 1 to add, 0 to change, 1 to destroy.
```

This indicated that Terraform planned to replace the affected resource.

Step 5: Apply the configuration

```
terraform apply
```

Review the plan and approve the operation when prompted.

The recorded result was:

```
Apply complete! Resources: 1 added, 0 changed, 1 destroyed.
```

Step 6: Verify the web endpoint

`curl -I http://localhost:8082`

The confirmed response was:

`HTTP/1.1 200 OK`

This confirmed that the web endpoint was responding through port `8082`.

Step 7: Inspect Terraform state

`terraform state list`

This lists the resources recorded in Terraform state. Insert the actual state output from the lab.

Step 8: Destroy the resources

`terraform destroy`

Review the proposed destruction and confirm it when prompted.

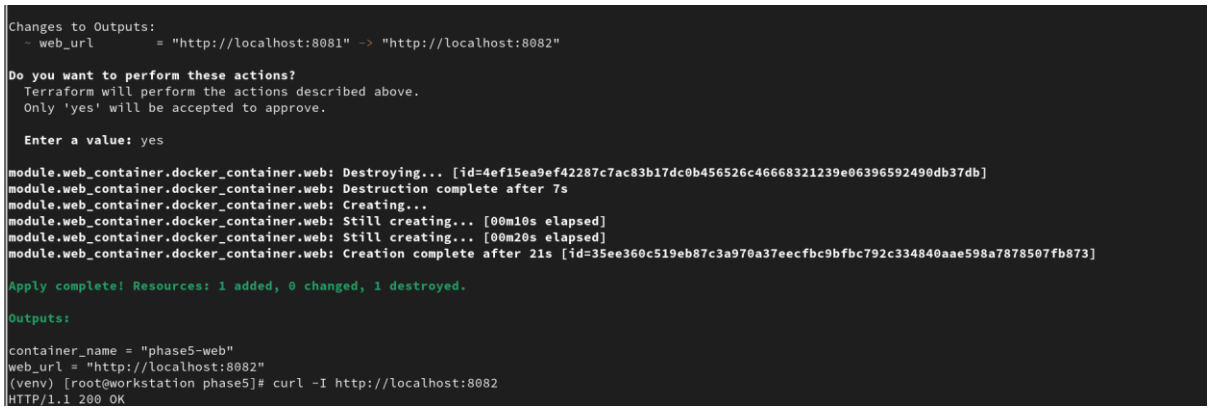
6.4 Result

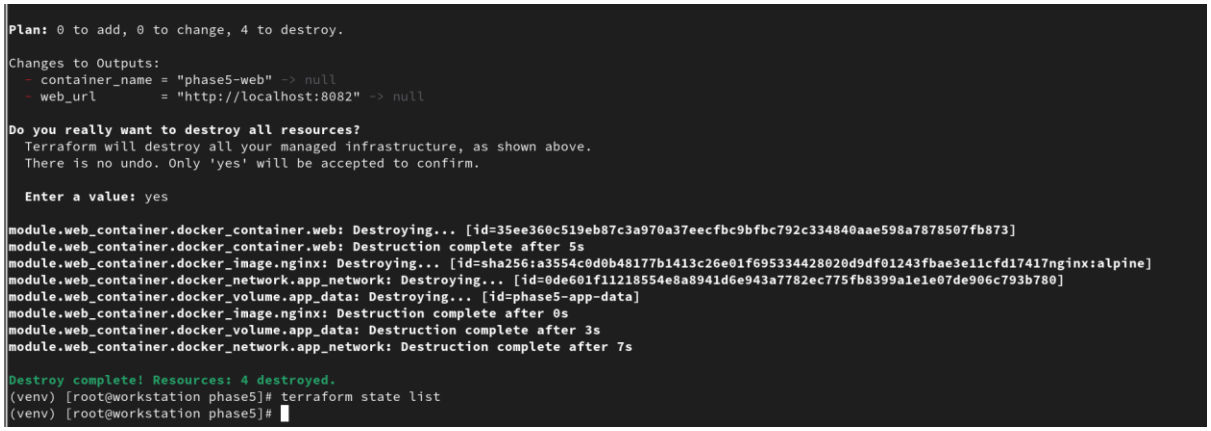
Terraform was used to initialize, plan, apply, inspect, and destroy infrastructure resources. Changing the host port from `8081` to `8082` resulted in the replacement of the affected resource. The web endpoint returned `HTTP/1.1 200 OK`.

The Terraform state used in this exercise was local; a remote backend was not configured.




root@workstation:/opt/devops-practical-lab/phase5
~ labels (known after apply)
~ ports {
~ external = 8081 -> 8082 # forces replacement
(3 unchanged attributes hidden)
}
- volumes {
- container_path = "/var/phase5-data" -> null
- read_only = false -> null
- volume_name = "phase5-app-data" -> null
(2 unchanged attributes hidden)
}
+ volumes {
+ container_path = "/var/phase5-data"
+ volume_name = "phase5-app-data"
(2 unchanged attributes hidden)
}
(1 unchanged block hidden)
}
Plan: 1 to add, 0 to change, 1 to destroy.
Changes to Outputs:
~ web_url = "http://localhost:8081" -> "http://localhost:8082"


Changes to Outputs:
~ web_url = "http://localhost:8081" -> "http://localhost:8082"
Do you want to perform these actions?
Terraform will perform the actions described above.
Only 'yes' will be accepted to approve.
Enter a value: yes
module.web_container.docker_container.web: Destroying... [id=4ef15ea9ef42287c7ac83b17dc0b456526c46668321239e06396592490db37db]
module.web_container.docker_container.web: Destruction complete after 7s
module.web_container.docker_container.web: Creating...
module.web_container.docker_container.web: Still creating... [00m10s elapsed]
module.web_container.docker_container.web: Still creating... [00m20s elapsed]
module.web_container.docker_container.web: Creation complete after 21s [id=35ee360c519eb87c3a970a37eecfbc9bfbc792c334840aae598a7878507fb873]
Apply complete! Resources: 1 added, 0 changed, 1 destroyed.
Outputs:
container_name = "phase5-web"
web_url = "http://localhost:8082"
(venv) [root@workstation phase5]# curl -I http://localhost:8082
HTTP/1.1 200 OK


Plan: 0 to add, 0 to change, 4 to destroy.
Changes to Outputs:
- container_name = "phase5-web" -> null
- web_url = "http://localhost:8082" -> null
Do you really want to destroy all resources?
Terraform will destroy all your managed infrastructure, as shown above.
There is no undo. Only 'yes' will be accepted to confirm.
Enter a value: yes
module.web_container.docker_container.web: Destroying... [id=35ee360c519eb87c3a970a37eecfbc9bfbc792c334840aae598a7878507fb873]
module.web_container.docker_container.web: Destruction complete after 5s
module.web_container.docker_image.nginx: Destroying... [id=sha256:a3554c0d0b48177b1413c26e01f695334428020d9df01243fbae3e11cfd17417nginx:alpine]
module.web_container.docker_network.app_network: Destroying... [id=0de601f11218554e8a8941d6e943a7782ec775fb8399a1e1e07de906c793b780]
module.web_container.docker_volume.app_data: Destroying... [id=phase5-app-data]
module.web_container.docker_image.nginx: Destruction complete after 0s
module.web_container.docker_volume.app_data: Destruction complete after 3s
module.web_container.docker_network.app_network: Destruction complete after 7s
Destroy complete! Resources: 4 destroyed.
(venv) [root@workstation phase5]# terraform state list
(venv) [root@workstation phase5]#

7. Phase 6 — Observability with Prometheus, Alertmanager, and Grafana

7.1 Objective

The sixth phase implemented monitoring for host and container resources. Prometheus collected metrics from Node Exporter and cAdvisor, evaluated a high CPU usage alert, and was configured to send alerts to Alertmanager. Grafana visualized the collected metrics in a dashboard.

7.2 Working Directory

```
cd /opt/devops-practical-lab/phase6
```

The Prometheus configuration directory was:

```
/opt/devops-practical-lab/phase6/prometheus
```

7.3 Prometheus Configuration: `prometheus/prometheus.yml`

```
global:
  scrape_interval: 15s
  evaluation_interval: 15s

rule_files:
  - /etc/prometheus/alert.rules.yml

alerting:
  alertmanagers:
    - static_configs:
        - targets:
            - alertmanager:9093

scrape_configs:
  - job_name: prometheus
    static_configs:
      - targets: ["prometheus:9090"]
```

```
  - job_name: node_exporter
    static_configs:
      - targets: ["node-exporter:9100"]

  - job_name: cadvisor
    static_configs:
      - targets: ["cadvisor:8080"]
```

Explanation

- Metrics are scraped every 15 seconds.
- Alert rules are loaded from `/etc/prometheus/alert.rules.yml`.
- Alertmanager is configured at `alertmanager:9093`.
- Prometheus scrapes itself, Node Exporter, and cAdvisor.
- Docker Compose service names provide internal service discovery.

7.4 Alert Rules: `prometheus/alert.rules.yml`

```
groups:
  - name: host_alerts
    rules:
      - alert: HighCPUUsage
        expr: 100 - (avg by (instance)
(rate(node_cpu_seconds_total{mode="idle"}[2m])) * 100) > 70
        for: 1m
        labels:
          severity: warning
        annotations:
          summary: "High CPU usage on {{ $labels.instance }}"
          description: "Host CPU usage has remained above 70% for
more than one minute."
```

Explanation

The rule calculates CPU utilization using the rate of idle CPU time reported by Node Exporter.

- The expression checks whether CPU usage exceeds 70%.
- The `for: 1m` setting requires the condition to remain true for one minute.
- The `severity` label identifies the alert as a warning.
- The annotations provide a summary and description.

7.5 Alertmanager Configuration: `alertmanager.yml`

```
global:
  resolve_timeout: 1m

route:
  receiver: lab-receiver
  group_by: ["alertname"]
  group_wait: 5s
  group_interval: 30s
  repeat_interval: 1h

receivers:
  - name: lab-receiver
```

This configuration defines the default alert receiver and grouping behavior. No external email or messaging integration was configured.

7.6 Docker Compose File: `docker-compose.yml`

```
services:
  prometheus:
    image: prom/prometheus:latest
    container_name: phase6-prometheus
    ports:
      - "9090:9090"
    volumes:

- ./prometheus/prometheus.yml:/etc/prometheus/prometheus.yml:ro

- ./prometheus/alert.rules.yml:/etc/prometheus/alert.rules.yml:ro
      - prometheus_data:/prometheus
    command:
      - "--config.file=/etc/prometheus/prometheus.yml"
      - "--storage.tsdb.path=/prometheus"
    depends_on:
      - node-exporter
      - cadvisor
      - alertmanager
    restart: unless-stopped

  node-exporter:
```

```
  image: prom/node-exporter:latest
  container_name: phase6-node-exporter
  ports:
    - "9100:9100"
  command:
    - "--path.rootfs=/host"
  volumes:
    - /:/host:ro,rslave
  pid: host
  restart: unless-stopped

cadvisor:
  image: gcr.io/cadvisor/cadvisor:latest
  container_name: phase6-cadvisor
  ports:
    - "8085:8080"
  privileged: true
  volumes:
    - /:/rootfs:ro
    - /var/run:/var/run:ro
    - /sys:/sys:ro
    - /var/lib/docker:/var/lib/docker:ro
    - /dev/disk:/dev/disk:ro
  restart: unless-stopped

alertmanager:
  image: prom/alertmanager:latest
  container_name: phase6-alertmanager
  ports:
    - "9093:9093"
  volumes:
    - ./alertmanager.yml:/etc/alertmanager/alertmanager.yml:ro
  command:
    - "--config.file=/etc/alertmanager/alertmanager.yml"
  restart: unless-stopped

grafana:
  image: grafana/grafana:latest
  container_name: phase6-grafana
  ports:
    - "3000:3000"
  volumes:
```

```
      - grafana_data:/var/lib/grafana
    environment:
      GF_SECURITY_ADMIN_USER: admin
      GF_SECURITY_ADMIN_PASSWORD: admin
    depends_on:
      - prometheus
    restart: unless-stopped

volumes:
  prometheus_data:
  grafana_data:
```

Explanation

The Compose file defines five services:

1. **Prometheus** collects and stores metrics and evaluates alert rules.
2. **Node Exporter** exposes host-level metrics.
3. **cAdvisor** exposes container metrics.
4. **Alertmanager** receives and routes alerts.
5. **Grafana** provides dashboards for the collected data.

Named volumes persist Prometheus and Grafana data. The monitoring services communicate over the default Compose network.

7.7 Deploy and Verify the Monitoring Stack

Validate the Compose file

```
docker compose config
```

This validates the Compose configuration and displays the resolved service definitions.

Start the services

```
docker compose up -d
```

The `-d` option starts the services in the background.

Verify the containers

```
docker compose ps
```

The five monitoring containers started successfully, and cAdvisor reported a healthy status.

7.8 Firewall Configuration

Started the firewall service and opened the required monitoring ports.

```
systemctl start firewalldfirewall-cmd --zone=public --add-port=9090/tcp --add-port=3000/tcp --add-port=9093/tcp --add-port=8085/tcp --permanentfirewall-cmd --reloadfirewall-cmd --zone=public --list-ports
```

The verified output included:

```
3000/tcp 8080/tcp 8085/tcp 9090/tcp 9093/tcp
```

The monitoring interfaces were accessed through the VM's IP address, `192.168.59.128`.

7.9 Verify Prometheus Targets

Opened:

http://192.168.59.128:9090

Navigated to **Status → Target health**.

Target	Purpose	Result
`prometheus`	Prometheus self-monitoring	UP
`node_exporter`	Host metrics	UP
`cadvisor`	Container metrics	UP

All three targets showed UP status.

7.10 Install and Run stress-ng

Checked package availability:

```
dnf info stress-ng
```

Installed the package:

```
dnf install -y stress-ng
```

Generated controlled CPU load:

```
stress-ng --cpu 2 --timeout 180s
```

This command runs two CPU stress workers for up to 180 seconds.

7.11 Verify the CPU Alert

Opened the Prometheus Alerts page and monitored `HighCPUUsage`.

The alert was initially INACTIVE. During the stress test, it reached FIRING. After the stress test ended and CPU utilization dropped below the threshold, it returned to INACTIVE.

7.12 Verify Alertmanager Connectivity

Opened the Alertmanager web interface at:

http://192.168.59.128:9093

Prometheus's Alertmanager API was accessed at:

http://192.168.59.128:9090/api/v1/alertmanagers

The returned response was:


```
{
  "status": "success",
  "data": {
    "activeAlertmanagers": [
      {
        "url": "http://alertmanager:9093/api/v2/alerts"
      }
    ],
    "droppedAlertmanagers": []
  }
}
```


This confirms that Prometheus reported an active Alertmanager endpoint and no dropped Alertmanager endpoints.

7.13 Configure Grafana

Opened:

http://192.168.59.128:3000

Logged into Grafana and added Prometheus as a data source.

The Prometheus server URL was:

http://prometheus:9090

This internal service address enables Grafana to communicate with Prometheus over the Compose network.

7.14 Grafana Dashboard Queries

Created and saved the dashboard:

Phase 6 - Infrastructure Monitoring

Panel 1: Host CPU Usage (%)

```
100 * (1 - avg(rate(node_cpu_seconds_total{mode="idle"}[5m])))
```

Visualization: Time series

Unit: Percent (0–100)

This query calculates CPU utilization by measuring the portion of CPU time spent doing work rather than remaining idle.

Panel 2: Host Memory Usage (%)

```
100 * (1 - node_memory_MemAvailable_bytes /
node_memory_MemTotal_bytes)
```

Visualization: Time series

Unit: Percent (0–100)

This query estimates host memory utilization using total memory and memory available to the system.

Panel 3: Active Container Count

```
count(container_last_seen{name!=""})
```

Visualization: Stat

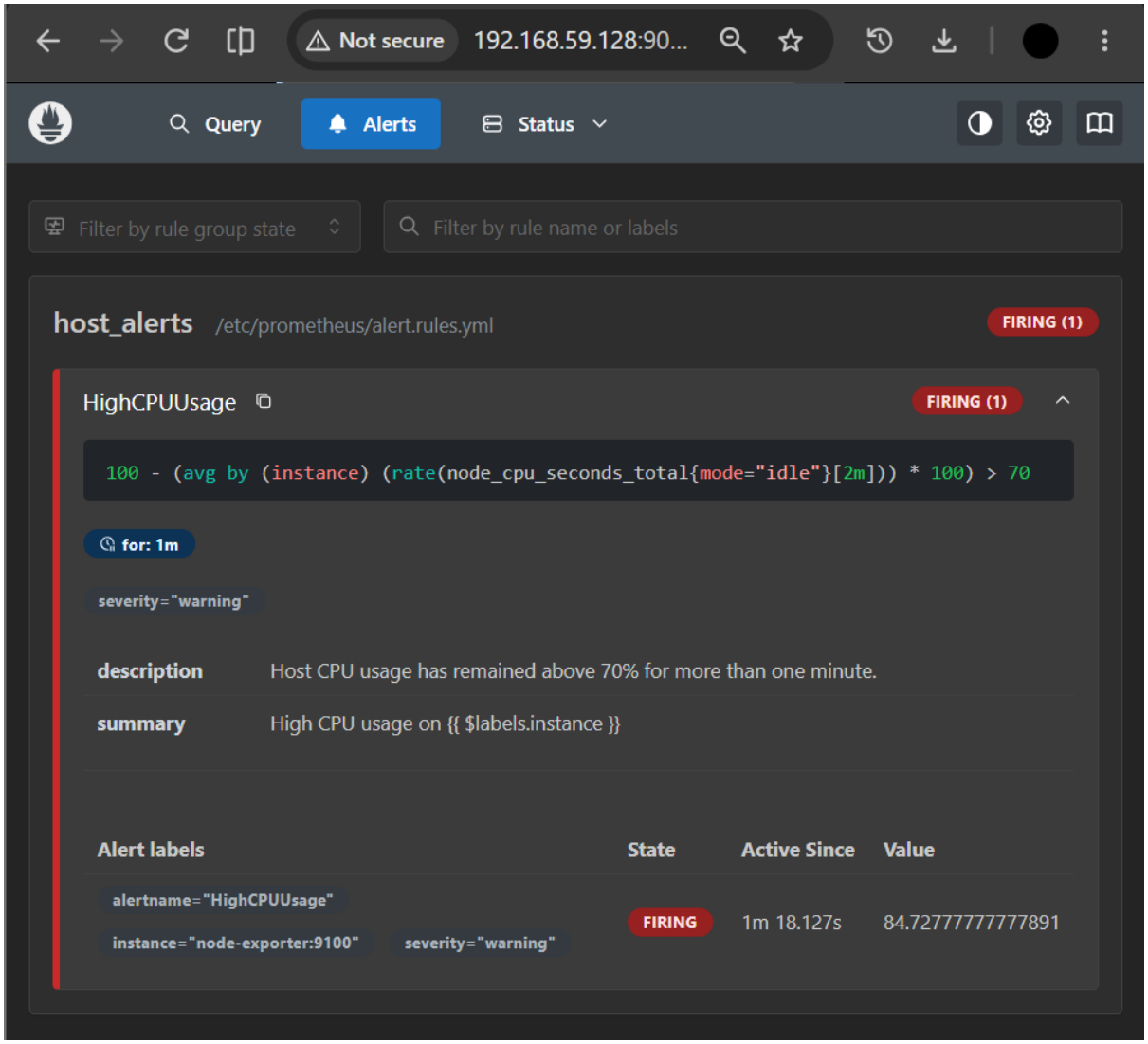
This query counts container time series exposed by cAdvisor with non-empty container names.

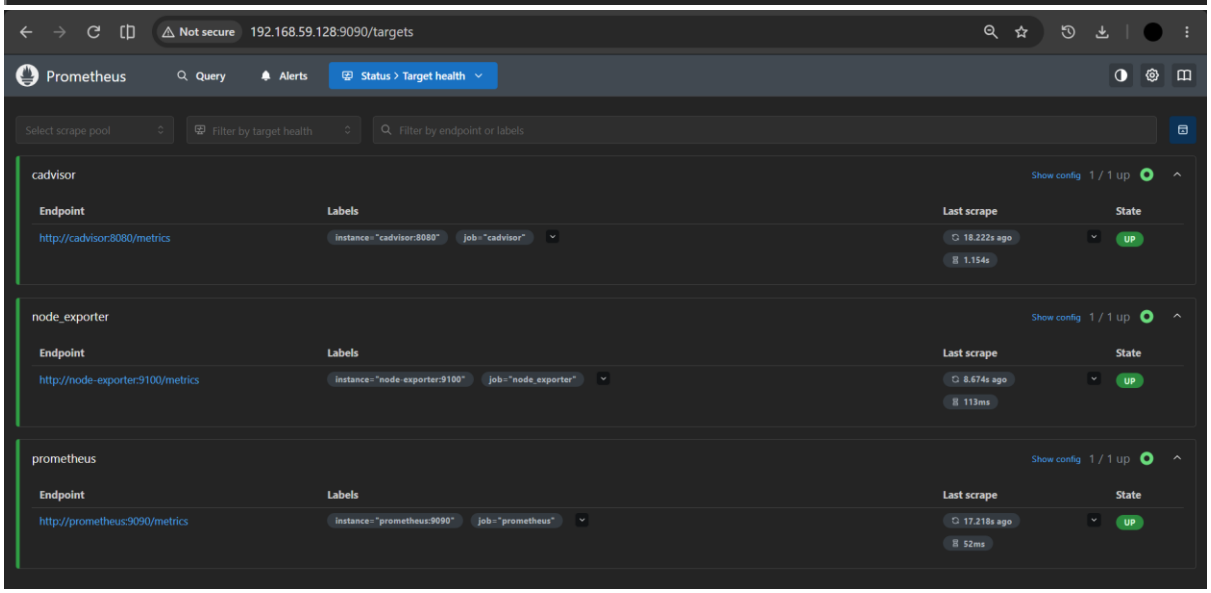
All three panels displayed metric data, and the dashboard was saved.

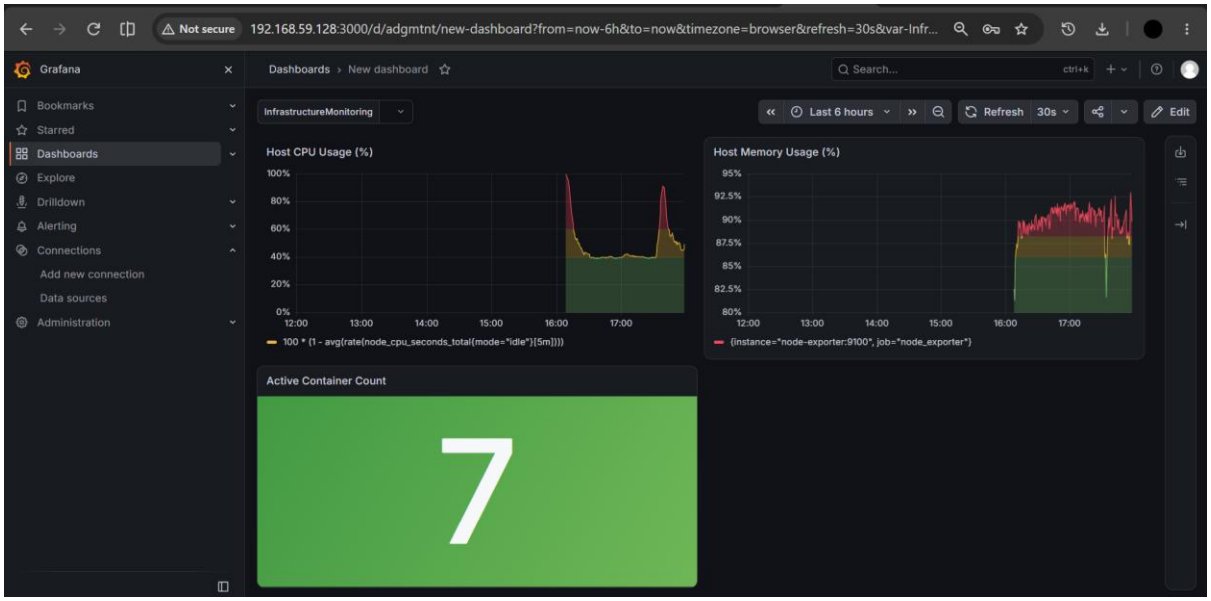
Screenshot 17: Insert the uncropped Grafana dashboard showing CPU usage, memory usage, and active container count together.

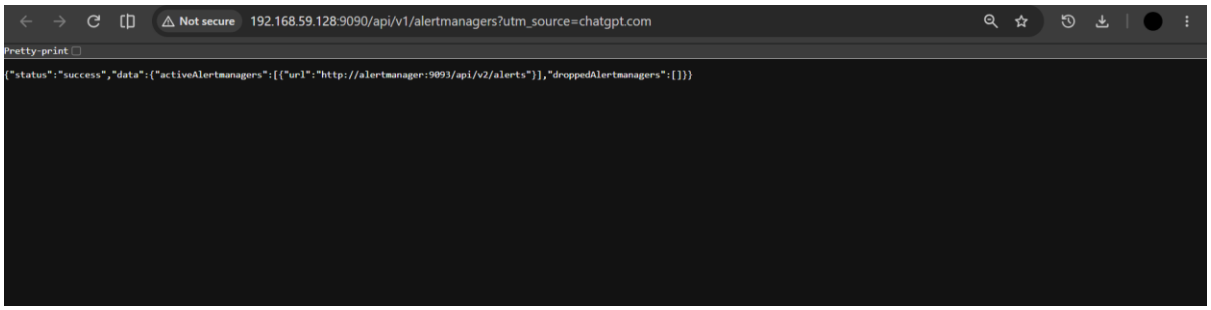
7.15 Result

The monitoring stack was deployed successfully. Prometheus collected host and container metrics, all three targets showed UP, and the high CPU alert reached FIRING during the stress test. The Alertmanager API showed an active endpoint, and Grafana displayed data across all three required panels.


Not secure
192.168.59.128:90...
Query
Alerts
Status
Filter by rule group state
Filter by rule name or labels
host_alerts
/etc/prometheus/alert.rules.yml
FIRING (1)
HighCPUUsage
FIRING (1)
100 - (avg by (instance) (rate(node_cpu_seconds_total{mode="idle"}[2m])) * 100) > 70
for: 1m
severity="warning"
description
Host CPU usage has remained above 70% for more than one minute.
summary
High CPU usage on {{ $labels.instance }}
Alert labels
State
Active Since
Value
alertname="HighCPUUsage"
instance="node-exporter:9100"
severity="warning"
FIRING
1m 18.127s
84.72777777777891


Not secure
192.168.59.128:9090/targets
Prometheus
Query
Alerts
Status > Target health
Select scrape pool
Filter by target health
Filter by endpoint or labels
cadvisor
Show config
1 / 1 up
Endpoint
Labels
Last scrape
State
http://cadvisor:8080/metrics
instance="cadvisor:8080"
job="cadvisor"
18.222s ago
1.154s
UP
node_exporter
Show config
1 / 1 up
Endpoint
Labels
Last scrape
State
http://node-exporter:9100/metrics
instance="node-exporter:9100"
job="node_exporter"
8.674s ago
113ms
UP
prometheus
Show config
1 / 1 up
Endpoint
Labels
Last scrape
State
http://prometheus:9090/metrics
instance="prometheus:9090"
job="prometheus"
17.218s ago
52ms
UP





8. Overall Conclusion

This six-phase DevOps practical project demonstrated application development, automated testing, containerization, CI/CD workflows, infrastructure lifecycle management, and infrastructure monitoring.

The early phases established a Python application and its testing workflow, followed by Docker packaging and automation using Jenkins and GitHub Actions. Terraform was used to plan and apply infrastructure changes and verify the updated web endpoint.

In Phase 6, Prometheus collected host and container metrics through Node Exporter and cAdvisor. A high CPU alert was tested with stress-ng, and Prometheus reported an active Alertmanager endpoint. Grafana provided real-time CPU, memory, and container-count visualization.

The project provided practical experience with Linux administration, containers, source control, automation pipelines, Infrastructure as Code, metrics collection, alert rules, and dashboard visualization.

The final document combines the implementation steps, relevant file contents, command logs, and verification evidence for all six phases.